

CopySight × Soneium

Proof of Creation & IP Validation — how CopySight's analysis results are recorded permanently on-chain, made independently verifiable by anyone, and protected against tampering.

Prepared for the CopySight team · September 2026

Overview

CopySight analyzes a file for copyright and IP matches. This integration writes a permanent, independently verifiable record of that analysis on Soneium's blockchain. Anyone holding the record's ID can look it up on-chain, indefinitely, and confirm: this exact file, analyzed this way, received this score, signed by a verified CopySight key — without needing to trust anyone's word for it, CopySight's included.

Soneium provides EAS (Ethereum Attestation Service) as a built-in part of the chain, so this integration doesn't invent a new data-storage mechanism — it uses the primitive the network already provides for exactly this kind of use case, with one purpose-built contract on top that enforces our own rules: who is allowed to sign, that scores fall within a valid range, and that the data is correctly shaped.

Build passing

177 automated tests

100% contract test coverage

Full on-chain flow proven end-to-end

How It Works



Every record carries four values on-chain: an asset fingerprint, an analysis fingerprint, a copy score, and an analysis-version fingerprint.

1. **The file is fingerprinted.** A cryptographic hash of the exact bytes — if even one pixel changes, the fingerprint is completely different. This is what "this exact file" means, provably.
2. **CopySight's analysis runs.** Its output — detected matches, similarity scores — is used to derive a single headline copy score (0–100). A clean asset with no matches scores 0.
3. **The analysis is fingerprinted too.** This is what "analyzed this way" means, provably — anyone can later confirm the analysis hasn't been altered.
4. **The record is submitted on-chain.** A purpose-built contract checks it on the way in — is this signer authorized, is the score valid, is the data correctly shaped. Any failed check reverts the transaction; nothing invalid is ever written.
5. **It's permanent.** Once written, anyone with the record's ID can look it up on Soneium directly and independently verify all of the above — no API call to CopySight required.

Key Concepts

EAS

Ethereum Attestation Service — a generic, signed-and-timestamped claim primitive, provided as a permanent part of Soneium itself.

Schema

The registered shape of the data in each record: an asset fingerprint, an analysis fingerprint, a copy score, and an analysis-version fingerprint.

Attestation

One actual record — one file's proof, with its own unique ID, signer, timestamp, and data.

Rule contract (resolver)

A smart contract that checks every record on the way in against our own rules — who's allowed to sign, valid score ranges, correct data shape.

Non-revocable, versioned corrections

Once written, a record can never be revoked or edited — by anyone, including CopySight. If an asset needs to be re-analyzed under a newer model version, a new record is submitted that explicitly references the prior one; the history stays intact.

Testnet vs. live network

Development and demonstration currently happen on Soneium's public testnet (Minato). The live network uses the same contracts and code, under separate configuration.

What's Been Built and Verified

Validated against real CopySight output

Real sample images and videos, run through CopySight's live analysis API — not synthetic test data. Correctly scored everything from clean assets to complex, multi-match content.

7 / 7 samples succeeded

The full on-chain lifecycle, proven end-to-end

Contract deployment, schema registration, record submission, confirmation, retrieval, and verification — proven against a live blockchain environment equivalent to the production network. The rule contract was also confirmed to correctly reject invalid submissions: an unauthorized signer, and an out-of-range score.

Fully automated, repeatable on demand

Database and persistence layer

Every record's lifecycle (submitted, confirmed, failed) is tracked in a real database, validated against an actual running instance — not just in isolation.

Fully automated, repeatable on demand

The smart contract itself

Every line, branch, and function of the on-chain rule contract is exercised by automated tests, including every failure case.

100% test coverage

Design Decisions

Record shape

Four fields: an asset fingerprint, an analysis fingerprint, a copy score, and an analysis-version fingerprint — deliberately minimal, matching the intended production design.

Purpose-built rule contract

Records are governed by a custom contract enforcing signer authorization, score validity, and data integrity — not the bare attestation service alone.

Immutable by design, versioned by design

Records can never be edited or revoked, matching the "permanent proof" positioning. Re-analysis under a newer model version is fully supported: a new record is submitted referencing the prior one, so the full history remains intact and inspectable. The analysis version itself is already tracked on every record; if a human-readable version identifier needs to be visible directly on-chain (rather than only in fingerprinted form), that's a small, straightforward addition — happy to scope it together.

Current Status

The integration is functionally complete and has been validated end-to-end against an environment equivalent to the live network. The one remaining step to go live on Soneium's public testnet is funding the wallet that deploys the contract and signs records — an access step, not a technical one.

NEXT STEP

Once testnet funds are available, deployment and registration are same-day tasks — every step of that process has already been proven working, not just written.

Questions You Might Have

Is a record really immutable — could it be altered later, by anyone?

No. Once written, a record cannot be edited or revoked by anyone, including CopySight — this is enforced by the attestation service's own core logic, not merely by convention.

What happens if a re-analysis later produces a different result?

A new record is submitted that explicitly references the original. Nothing is hidden or overwritten — both records remain visible, with a clear link between them.

How is the system protected against fraudulent records?

The rule contract checks every submission against an on-chain allow-list of authorized signers. Anything from an unrecognized address is rejected automatically, before it's ever written.

What does this cost to operate?

Each record is a single, small transaction on an L2 network — a small fraction of a cent to a few cents depending on network conditions. Setup steps (initial contract deployment, schema registration) are one-time costs.

Does this support both images and video?

Yes — validated against real samples of both, through the same code path.

Has the contract been independently audited?

Not yet by a third-party firm — that's a standard pre-launch step for any production deployment. It currently carries 100% automated test coverage, including every failure case, and has been validated against a live-equivalent network end-to-end.

Is this locked to Soneium, or portable to another chain later?

The core logic never depends on Soneium specifics directly — it's built against a generic interface, with Soneium as one implementation of it. Supporting an additional chain later is an additive change, not a rewrite.

What happens if a submission fails partway through?

Every record's status is tracked and persisted. Transient network issues are retried automatically; a genuine rejection (such as an unauthorized signer) fails immediately rather than retrying something that can never succeed. If confirmation is delayed, the system checks whether the record actually landed before assuming failure — preventing accidental duplicate submissions.

Can a third party verify a record without involving CopySight?

Yes — that's the purpose of putting it on a public chain. Anyone with a record's ID can query Soneium directly and see the data, the signer, and the timestamp.

Does resubmitting the same asset create duplicate records?

No — each request is keyed deterministically. A repeat request for identical content returns the existing record rather than creating a new one.